

**LAPORAN PRAKTIKUM**  
**PEMROGRAMAN BERORIENTASI OBJEK**  
**MODUL IV & V**  
**CLASS dan OBJECT**



**Disusun Oleh:**  
**Arif Mufti Tharsa 105224041**

**PROGRAM STUDI ILMU KOMPUTER**  
**FAKULTAS SAINS DAN KOMPUTER**  
**UNIVERSITAS PERTAMINA**  
**2026**

## **I. Pendahuluan**

Di THT Modul 3 & 4 ini, saya membuat program Java bernama KamarHotel.java yang mensimulasikan sistem manajemen kamar hotel sederhana. Materi yang diterapkan di sini adalah Class dan Object, yaitu konsep dasar dalam Pemrograman Berorientasi Objek (OOP) yang mencakup pembuatan class, deklarasi atribut, penggunaan constructor, serta pendefinisian method untuk mengatur perilaku objek.

Secara umum, program ini dirancang untuk merepresentasikan sebuah kamar hotel sebagai objek nyata dalam dunia pemrograman. Program dapat membuat objek kamar, mengatur dan memvalidasi datanya, memesan atau membatalkan pemesanan kamar, menghitung total biaya menginap, hingga menampilkan informasi lengkap kamar ke konsol. Semua fungsionalitas tersebut dikemas menggunakan prinsip encapsulation dan method overloading dalam satu class yang terstruktur.

## II. Variabel

| No | Nama Variabel     | Tipe Data | Fungsi                                                                |
|----|-------------------|-----------|-----------------------------------------------------------------------|
| 1  | nomorKamar        | String    | Menyimpan nomor identitas kamar hotel                                 |
| 2  | tipeKamar         | String    | Menyimpan tipe kamar (Reguler, Premium, Suite)                        |
| 3  | kapasitasMaksimal | int       | Menyimpan batas maksimal jumlah tamu yang diperbolehkan               |
| 4  | hargaPerMalam     | double    | Menyimpan harga sewa kamar per malam dalam rupiah                     |
| 5  | isTersedia        | boolean   | Menyimpan status ketersediaan kamar (true = tersedia, false = terisi) |

### III. Constructor dan Method

| No | Nama Metode                             | Jenis Metode        | Fungsi                                                                 |
|----|-----------------------------------------|---------------------|------------------------------------------------------------------------|
| 1  | KamarHotel(String, String, int)         | Constructor         | Membuat objek kamar dengan harga default 0 dan status tersedia         |
| 2  | KamarHotel(String, String, int, double) | Constructor         | Membuat objek kamar dengan harga yang ditentukan langsung              |
| 3  | getNomorKamar()                         | Functional (getter) | Mengembalikan nilai nomorKamar                                         |
| 4  | getTipeKamar()                          | Functional (getter) | Mengembalikan nilai tipeKamar                                          |
| 5  | getKapasitasMaksimal()                  | Functional (getter) | Mengembalikan nilai kapasitasMaksimal                                  |
| 6  | getHargaPerMalam()                      | Functional (getter) | Mengembalikan nilai hargaPerMalam                                      |
| 7  | getStatus()                             | Functional (getter) | Mengembalikan status ketersediaan kamar                                |
| 8  | setTipeKamar(String)                    | Procedural (setter) | Mengatur tipeKamar dengan validasi; jika tidak valid diubah ke Reguler |
| 9  | setHargaPerMalam(double)                | Procedural (setter) | Mengatur harga dengan validasi minimum Rp50.000                        |
| 10 | pesanKamar()                            | Procedural          | Memesan kamar tanpa cek jumlah tamu                                    |

|    |                               |            |                                                                           |
|----|-------------------------------|------------|---------------------------------------------------------------------------|
| 11 | pesanKamar(int)               | Procedural | Memesan kamar dengan validasi jumlah tamu terhadap kapasitas maksimal     |
| 12 | batalPesan()                  | Procedural | Membatalkan pemesanan dan mengembalikan status kamar menjadi tersedia     |
| 13 | hitungTotalBayar(int)         | Functional | Menghitung total biaya berdasarkan jumlah malam tanpa diskon              |
| 14 | hitungTotalBayar(int, String) | Functional | Menghitung total biaya dengan kemungkinan diskon menggunakan kode voucher |
| 15 | tampil()                      | Procedural | Menampilkan seluruh informasi kamar ke konsol                             |

#### IV. Dokumentasi dan Pembahasan Code

```

public class KamarHotel {
    private String nomorKamar, tipeKamar;

    private int kapasitasMaksimal;

    private double hargaPerMalam;

    private boolean isTersedia;

    public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal) {
        this.nomorKamar = nomorKamar;
        setTipeKamar(tipeKamar);
        this.kapasitasMaksimal = kapasitasMaksimal;
        this.hargaPerMalam = 0;
    }
}

```

```
        this.isTersedia = true;
    }

    public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal,
double hargaPerMalam) {

        this.nomorKamar = nomorKamar;

        setTipeKamar(tipeKamar);

        this.kapasitasMaksimal = kapasitasMaksimal;

        setHargaPerMalam(hargaPerMalam);

        this.isTersedia = true;
    }

    public String getNomorKamar() {

        return nomorKamar;
    }

    public String getTipeKamar() {

        return tipeKamar;
    }

    public int getKapasitasMaksimal() {

        return kapasitasMaksimal;
    }

    public double getHargaPerMalam() {

        return hargaPerMalam;
    }

    public boolean getStatus() {

        return isTersedia;
    }
}
```

```

public void setTipeKamar(String tipeKamar) {

    if (tipeKamar.equalsIgnoreCase("Reguler") ||
tipeKamar.equalsIgnoreCase("Premium") || tipeKamar.equalsIgnoreCase("Suite")){

        this.tipeKamar = tipeKamar;

    } else {

        System.out.println("Tipe tidak valid, diubah ke Reguler");

        this.tipeKamar = "Reguler";

    }

}

public void setHargaPerMalam(double hargaPerMalam) {

    if (hargaPerMalam < 50000){

        System.out.println("Harga tidak valid, diubah ke 50000");

        this.hargaPerMalam = 50000;

    } else {

        this.hargaPerMalam = hargaPerMalam;

    }

}

public void pesanKamar() {

    if (isTersedia) {

        isTersedia = false;

        System.out.println("Kamar berhasil dipesan");

    } else {

        System.out.println("Kamar sudah terisi");

    }

}

public void pesanKamar(int jumlahTamuh) {

    if (!isTersedia){

        System.out.println("Kamar sudah terisi");

    } else if (jumlahTamuh > kapasitasMaksimal){

```

```

        System.out.println("Tamu melebihi kapasitas!");
    } else {

        isTersedia = false;

        System.out.println("Berhasil pesan untuk " + jumlahTamu + " tamu");
    }
}

public void batalPesan(){

    isTersedia = true;
}

public double hitungTotalBayar(int malam){

    return malam * hargaPerMalam;
}

public double hitungTotalBayar(int malam, String kode){

    double total = malam * hargaPerMalam;

    if (kode.equalsIgnoreCase("PROMO") && malam >= 3){

        System.out.println("Diskon 20%");

        return total * 0.8;
    } else {

        System.out.println("Voucher gagal");

        return total;
    }
}

public void tampil(){

    System.out.println("\n--- Struk ---");

    System.out.println("Nomor: " + nomorKamar);

    System.out.println("Tipe: " + tipeKamar);
}

```

```

        System.out.println("Kapasitas: " + kapasitasMaksimal);

        System.out.println("Harga: " + hargaPerMalam);

        System.out.println("Status: " + (isTersedia ? "Tersedia" : "Terisi"));
    }
}

public class MainKamarHotel {

    public static void main(String[] args) {

        KamarHotel kamar1 = new KamarHotel("101", "Presidential", 3, -10000);

        kamar1.setTipeKamar("Presidential");

        kamar1.setHargaPerMalam(-10000);

        KamarHotel kamar2 = new KamarHotel("102", "Premium", 2);

        kamar2.pesanKamar(4);

        kamar2.pesanKamar(2);

        kamar2.pesanKamar();

        System.out.println("\n--- Total pembayaran ---");

        double total1 = kamar1.hitungTotalBayar(2, "PROMO");

        System.out.println("Total kamar1: " + total1);

        double total2 = kamar2.hitungTotalBayar(4, "PROMO");

        System.out.println("Total kamar2: " + total2);

        kamar1.tampil();

        kamar2.tampil();

    }
}

```

Penjelasan dari program KamarHotel:

Class KamarHotel memiliki lima atribut utama, yaitu nomorKamar dan tipeKamar bertipe String untuk identitas kamar, kapasitasMaksimal bertipe int untuk batas jumlah tamu, hargaPerMalam bertipe double untuk harga, dan isTersedia bertipe boolean untuk status ketersediaan kamar. Semua atribut dideklarasikan sebagai private, yang artinya tidak bisa diakses langsung dari luar class. Ini

adalah penerapan encapsulation supaya data kamar lebih aman dan terkontrol, hanya bisa diakses atau diubah lewat getter dan setter.

Class ini memiliki dua constructor dengan jumlah parameter yang berbeda, yang merupakan contoh constructor overloading. Constructor pertama hanya menerima tiga parameter (nomorKamar, tipeKamar, kapasitasMaksimal) dan secara otomatis mengatur harga ke 0. Constructor kedua menerima satu parameter tambahan yaitu hargaPerMalam. Menariknya, kedua constructor tidak langsung mengassign nilai tipeKamar dan harga secara langsung, melainkan memanggil setter-nya terlebih dahulu agar validasi tetap berjalan sejak awal pembuatan objek.

Setter setTipeKamar dan setHargaPerMalam adalah dua method yang memiliki validasi di dalamnya. setTipeKamar mengecek apakah tipe yang dimasukkan termasuk "Reguler", "Premium", atau "Suite" dengan ignoreCase agar tidak case-sensitive. Kalau tidak sesuai, tipeKamar otomatis diubah ke "Reguler". Sementara setHargaPerMalam memastikan harga tidak boleh kurang dari Rp50.000; jika di bawah itu maka harga langsung diganti jadi 50.000. Ini penting supaya data yang tersimpan dalam objek selalu masuk akal dan tidak menyebabkan error di bagian lain program.

Method pesanKamar juga menggunakan overloading. Versi tanpa parameter langsung memesan kamar tanpa mengecek jumlah tamu, sedangkan versi dengan parameter jumlahTamu terlebih dahulu memvalidasi apakah jumlah tamu melebihi kapasitas. Begitu pula dengan hitungTotalBayar, yang punya dua versi: satu hanya menghitung malam dikali harga, satunya lagi menerima kode voucher dan memberikan diskon 20% jika kodenya "PROMO" dan minimal menginap 3 malam.

Di class MainKamarHotel, program membuat dua objek kamar sebagai pengujian. kamar1 dibuat dengan tipe "Presidential" dan harga -10.000, keduanya tidak valid sehingga tipe otomatis menjadi "Reguler" dan harga diubah menjadi 50.000. kamar2 dibuat tanpa harga lalu langsung dicoba dipesan untuk 4 tamu, namun ditolak karena melebihi kapasitas 2. Setelah itu berhasil dipesan untuk 2 tamu, kemudian dicoba dipesan lagi tapi sudah terisi. Di bagian akhir, total bayar keduanya dihitung dan seluruh informasi kamar ditampilkan melalui method tampil().

## **V. Kesimpulan**

1. Encapsulation menjaga data atribut tetap aman lewat akses private dan kontrol melalui getter-setter. Dengan cara ini, data dalam objek tidak bisa sembarangan diubah dari luar class, sehingga integritas data selalu terjaga.
2. Method overloading terbukti berguna untuk memberikan fleksibilitas. Satu nama method bisa menangani skenario yang berbeda tanpa perlu membuat nama method baru, sehingga kode lebih bersih dan mudah dipahami.
3. Constructor overloading memudahkan pembuatan objek dengan tingkat kelengkapan data yang berbeda-beda sesuai kebutuhan, tanpa harus selalu menyediakan semua parameter sekaligus.
4. Validasi di dalam setter sangat penting untuk memastikan data yang masuk ke objek selalu valid. Tanpa validasi ini, objek bisa menyimpan data yang tidak logis dan menyebabkan kesalahan di bagian lain program.